

Gate Class Documentation

Source File: Gate.h
Namespace: cap
Class Header: class Gate : public Dev

Overview

The *Gate* abstract class is a *Dev* that can be an input for exactly one Boolean function and can take inputs within a range.

Constructors

- Gate() (default constructor)
 - **Purpose:** Creates a gate that accepts exactly one input.
- Gate(size_t sz)
 - **Purpose:** Creates a gate that accepts 1 upto *sz* inputs if *sz* is greater than 1 and at most 10; otherwise, use the default range.
 - **Parameter(s):**
 - *sz*: Unsigned integer.
- Gate(size_t i, size_t j)
 - **Purpose:** Creates a gate that accepts between *i* and *j* inputs if both parameters are greater than 1 and at most 10; otherwise, use the default range.
 - **Parameter(s):**
 - *i*: Unsigned integer.
 - *j*: Unsigned integer.
- Gate(const Gate& obj) (copy constructor) [deleted]

Destructor

- ~Gate() [virtual]
 - **Purpose:** Attempts to unbind inputs.

Assignment Operator

- operator=(const Gate& rhs) [deleted]

Member Functions

- bind(Dev& obj) [private]
- unbind() [private]
- at(size_t idx)
 - **Purpose:** Retrieves the child function with index *idx* if *idx* is a valid index.
 - **Parameter(s):**
 - *idx*: Unsigned integer.
 - **Exception(s):**
 - out_of_range: Thrown if *idx* exceeds the number of inputs.
 - **Return:** Constrant *Dev* reference.
- broken() const [final]
 - **Purpose:** Returns true if it is broken or any of its inputs are broken; otherwise, it returns false.
 - **Return:** Boolean.
- reset() [final]
 - **Purpose:** Removes its inputs and parent function.
- input(Dev& obj)
 - **Purpose:** Makes *obj* an input and returns true only if it is not broken or full and *obj* is not bounded or bounded to it; otherwise, it returns false.
 - **Parameter(s):**
 - *obj*: *Dev* reference object.

- `count()` `const`
 - **Purpose:** Returns the number of inputs.
 - **Return:** Unsigned integer.
- `valid()` `const` `[final]`
 - **Purpose:** Returns true only if it is not broken and has its minimum required inputs.
 - **Return:** Boolean.
- `full()` `const`
 - **Purpose:** Returns true if it is the maximum number of inputs.
 - **Return:** Boolean.
- `minimum()` `const`
 - **Purpose:** Returns the minimum number of inputs required.
 - **Return:** Boolean.
- `maximum()` `const`
 - **Purpose:** Returns the maximum number of inputs required.
 - **Return:** Boolean.